

COL724/7524 Assignment 2, Part A: Link Scheduling

Entry numbers: 2026MCS2494, 2026MCS2493

1. The four policies as implemented

Queue and threads (A5). An acceptor thread only calls `accept()`. Each connection goes to a short-lived admission thread that reads the header line under a 5 s total deadline, answers HEALTH immediately with the current queue depth, validates the file name (A25), and enqueues the request. For a GET it opens the file and takes the size from `fstat`, so the size is known, and fixed to one version of the file, when the request enters the queue. *server_threads* workers only loop on `next()`, then `serve_slice()` for one round, then either requeue the request (PREEMPTED) or finish it. Because admission never waits for a worker, requests really do pile up in the scheduler queue: a test that pins the only worker on a stalled PUT sees HEALTH report a depth of 6, and HEALTH still answers in about 1 ms while a silent client is connected. Clients that trickle bytes are cut off by total deadlines (5 s for the header, 10 s per 64 KB of a PUT body), not by a `per-recv()` timeout, so they cannot pin an admission thread or a worker either.

Policies. *fcfs* serves in arrival order. *sjf* serves the queued request with the smallest declared byte count (file size for GET, the count in the request line for PUT: one key for both verbs), earliest arrival first on ties, with no aging. *fcfs* and *sjf* never preempt, so every request takes one round (*rounds* = 1). *rr* gives each scheduled request an allowance of $Q = 8192$ bytes per round and requeues it at the tail. *drr* uses the same queue but each request carries a deficit counter (initially 0): a round's allowance is deficit + Q , and whatever part of it goes unused is kept in the deficit instead of being forfeited; the deficit is discarded when the request finishes.

One round (*serve_slice*). A GET is sent in whole lines (A6): the line length is found by scanning to the next '\n' (a last line without one still counts), and lines are grouped `--p` per `write()` (A8) without changing which bytes go out or how the allowance is charged. A round ends as soon as the next line would exceed the remaining allowance (A13); under *rr* the unused remainder is added to *forfeited_bytes*, under *drr* it becomes the deficit. **A14** (*rr* only): if a line longer than Q starts a round, i.e. the round has transferred nothing yet, it is sent in full and the round ends, overrunning the allowance and logging one line. I read A14 this way because its stated purpose is to stop a request being requeued having transferred nothing, forever; a long line reached *mid-round* is handled by A13 instead (round ends, remainder forfeited) and goes out first in the next round. *drr* has no escape: the deficit grows by Q per round until it covers the line (A16), so a 20,000-byte line costs up to 3 rounds. A PUT round reads exactly $\min(\text{allowance}, \text{remaining})$ bytes (A9, A13) into a temporary file that is renamed onto the destination when the last byte arrives, so a concurrent GET never sees a half-written file. The 'OK n' response line is protocol overhead and is not charged to the allowance; the terminating '\n' of every line is (A7).

Preemption state (A17). Everything a later round needs lives in the Request: the byte offset; the file descriptor opened at admission (so every round reads the version that was sized); the bytes already read from the socket but not yet consumed (for a PUT, the body bytes that arrived with the header); the deficit; *rounds* and *forfeited_bytes*; and the temporary-file path. Correctness is checked by 72 unit assertions that drive *serve_slice* over a message-preserving socketpair against rounds, forfeiture and A14 counts worked out by hand (e.g. ten 10-byte lines with $Q = 35$: *rr* takes 4 rounds and forfeits 15 bytes, *drr* takes 3 and forfeits 0), including a GET that keeps serving the old file after a PUT has replaced it, and by an integration suite that runs all four policies end to end.

2. Experimental setup

Machine and configuration. Server and client run on one laptop under WSL2 (Ubuntu 24.04, 8 logical CPUs) over loopback TCP, so absolute times are not network-representative; only comparisons between policies are meaningful. The reference configuration is 4 server threads and 8 client threads; the second is 1 server thread. $Q = 8192$ bytes. Each run makes 1000 counted requests from a closed loop of 8 client threads, after 3 uncounted seeding PUTs (dropped by sorting the CSV by arrival and removing the first 3 rows). Each request picks one of the workload files uniformly and does a GET or a PUT with probability 1/2. Percentiles are nearest-rank. `TCP_NODELAY` is set so every `write()` is a separate segment.

Workload (A27), both axes in one directory. *Size*: small.txt 1,044 B, medium.txt 30,774 B, large.txt 153,613 B. *Line length*: small and medium contain only ordinary 60-79 byte lines; large.txt has the same ordinary lines plus 6 lines of 20,000 bytes ($2.4 \times Q$) spread evenly through it, so it is both the large file and the long-line file. With $Q = 8192$ the large file needs 19 rounds under a byte-exact accounting.

Repetition. A28 asks for one run per cell. This setup is noisy from run to run (background activity on the Windows host made the slowest run of a cell up to 2 times slower than its fastest), so every cell was run **5 times**, in interleaved blocks (each block runs all cells once, preceded by an unrecorded warm-up run because the first cell of a session otherwise pays a cold-start cost). Tables report the **median over the 5 runs**, with [min-max] where useful; whiskers in the figures are min-max. Raw per-request CSVs of every run are in results/run1..run5. Two extra cells, *sjf* and *drr* at 1 server thread, are supplementary (not among the six required cells) and are marked as such.

3. Per-run results

Table 1. Waiting time (start - arrival), response time (finish - arrival) and throughput, median [min-max] over 5 runs.

cell	waiting p50 (us)	waiting p99 (us)	response p50 (us)	response p99 (us)	throughput (req/s)
fcfs, 4 thr	1082 [891-1360]	5873 [4154-8490]	2654 [2279-3385]	12515 [10651-13889]	1776 [1551-2011]
sjf, 4 thr	842 [752-948]	9540 [8938-15649]	2833 [2312-3502]	17810 [15372-30672]	1497 [1250-1667]
rr, 4 thr	283 [258-354]	2257 [1463-3978]	3187 [2803-4712]	16924 [14055-36955]	1291 [985-1553]
drr, 4 thr	257 [234-279]	1866 [1282-3859]	3200 [2673-3587]	17124 [16137-18403]	1390 [1146-1512]
fcfs, 1 thr	16277 [13397-22198]	40710 [29134-96055]	19004 [15955-25702]	46548 [33417-99781]	405 [268-488]
rr, 1 thr	2351 [2298-2537]	6739 [6323-11600]	11678 [11127-14056]	71878 [64761-77147]	436 [394-442]
drr, 1 thr (suppl.)	2235 [2182-2585]	7942 [5968-9399]	12124 [11221-12684]	73530 [60945-76822]	403 [374-415]
sjf, 1 thr (suppl.)	5299 [4826-5977]	71161 [69850-99168]	6191 [5868-7692]	75616 [71861-107337]	417 [363-470]

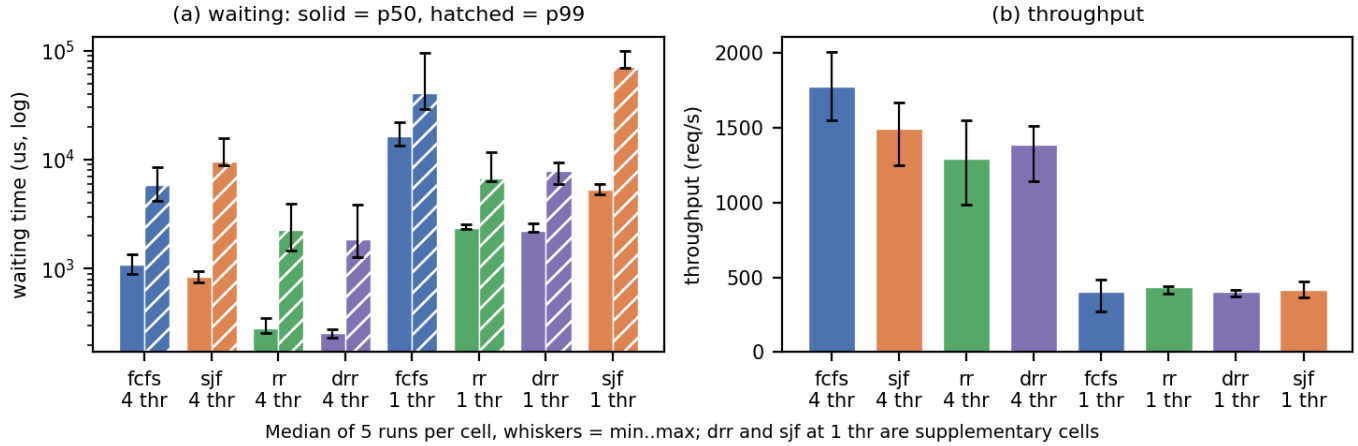
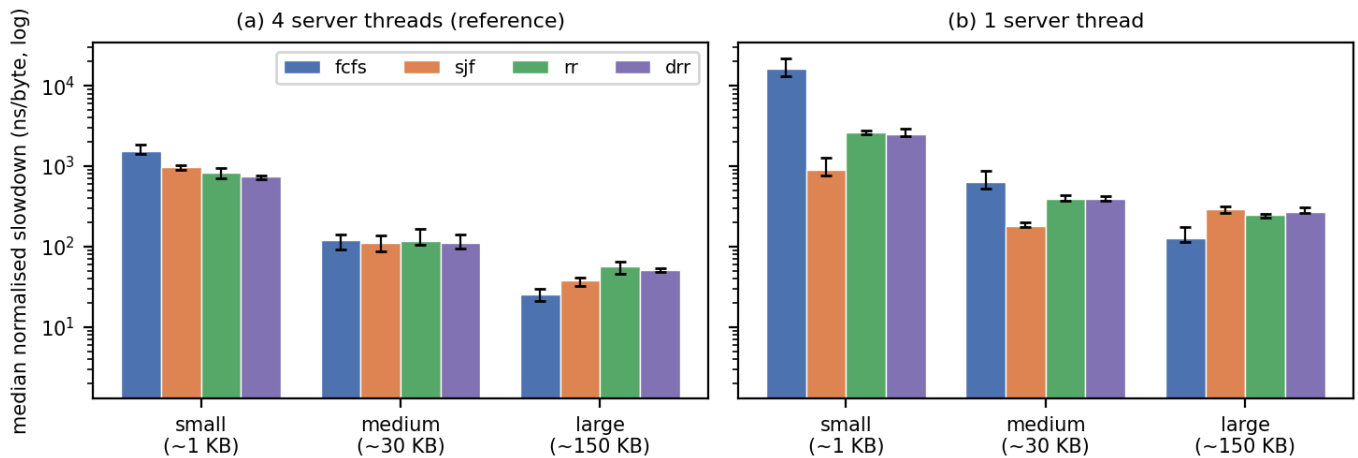


Table 2. Normalised slowdown (response / bytes, ns per byte) by size class: median over runs of each run's median and p99. The first four rows are the reference configuration.

cell	small med	small p99	medium med	medium p99	large med	large p99
fcfs, 4 thr	1546	6155	119	391	25	88
sjf, 4 thr	979	8746	112	571	38	136
rr, 4 thr	826	6753	117	433	57	121
drr, 4 thr	737	4837	112	439	52	130
fcfs, 1 thr	16234	44586	638	1403	126	294
rr, 1 thr	2641	6627	390	863	250	499
drr, 1 thr (suppl.)	2482	7995	389	827	271	573
sjf, 1 thr (suppl.)	910	9004	182	539	294	563



Reading the numbers. Median waiting falls from 1082 us under fcfs to 842 us under sjf and to 283 / 257 us under rr / drr (about 3.8x lower), and with one server thread from 16.3 ms to 2.4 ms under rr: the preemptive policies keep the queue short for everyone because no request holds a worker for its whole transfer. Waiting is what the policy controls; the slowdown table shows who pays for it. The small class gains most: median slowdown 1546 (fcfs) to 979 (sjf) to 826 / 737 ns/byte (rr / drr), and with one server thread 16234 to 2641. The large class pays: 25 (fcfs), 38 (sjf), 57 / 52 (rr / drr); with one thread 126 under fcfs against 250 under rr. Medium files barely move at the

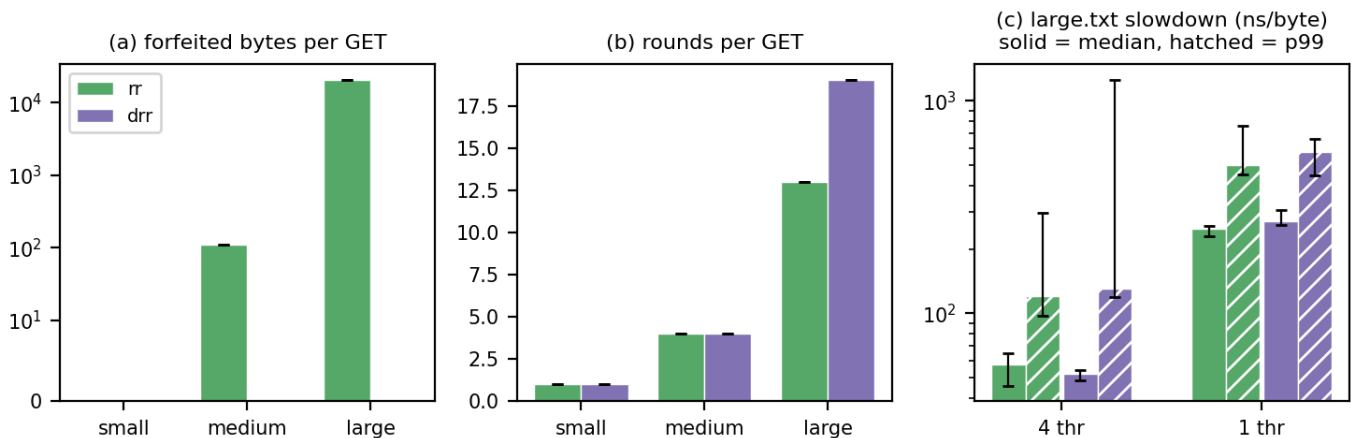
reference configuration and gain under one thread. Throughput is lower under rr/drr (median 1291 / 1390 req/s vs 1776 for fcfs at the reference configuration): every extra round costs a queue trip and extra system calls. Medians are 27% (rr) and 22% (drr) below fcfs, but the run ranges overlap (fcfs 1776 [1551-2011], rr 1291 [985-1553], drr 1390 [1146-1512]), so the size of the cost is uncertain even though its direction is consistent. With one server thread the throughputs (405 fcfs vs 436 rr) are not separable: the single worker is the bottleneck either way.

Waiting versus response (A19). The two tell different stories. At the reference configuration rr / drr cut median waiting 3.8x, yet median response is not lower: 3187 / 3200 us against 2654 us for fcfs. Response also contains the request's own service time, and a preempted request's service is spread over several rounds interleaved with other requests, so the earlier start is spent on the way to the finish. With one server thread the queue is long enough for the gain to survive: median response falls from 19.0 ms (fcfs) to 11.7 ms (rr) and 6.2 ms (sjf), but the tail moves the other way: p99 response 47 ms (fcfs), 72 ms (rr), 76 ms (sjf), because the large requests now wait through many rounds or behind every smaller request. Response time alone would have hidden the first effect.

4. rr versus drr (A29)

Table 3. Long-line comparison, median over 5 runs. A14 fired 1026 times per rr run at the reference configuration and 1020 at one thread (range 882-1110 over all runs): exactly the 6 long lines of every large-file GET (about 171 GETs per run) and never under drr.

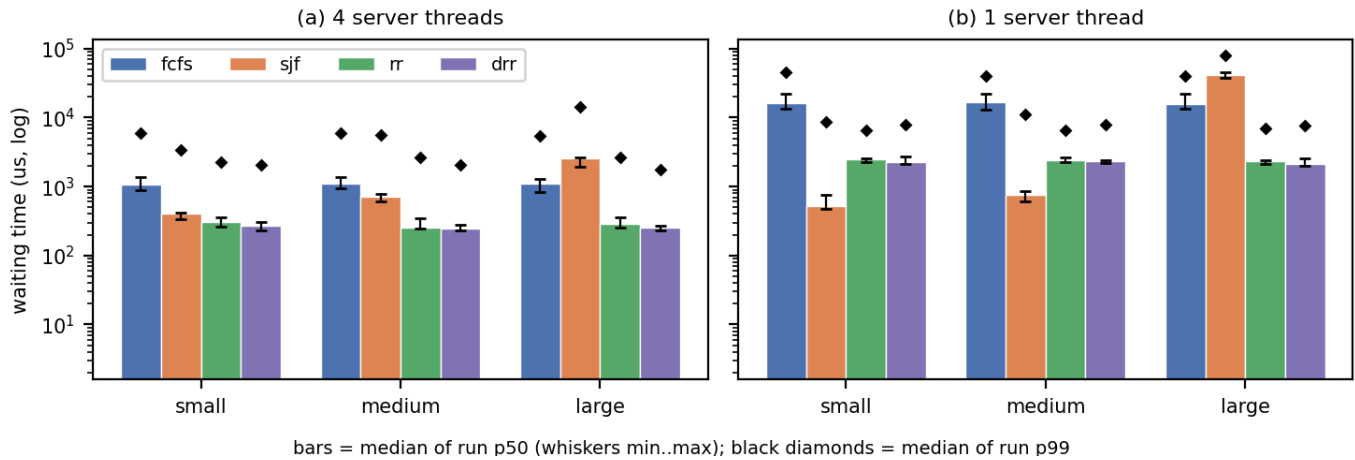
cell	forfeited bytes (all requests of a run)			rounds / GET		A14 fires	large.txt slowdown
	small	medium	large	medium	large	per run	median / p99
rr, 4 thr	0	19,140	3,469,419	4.0	13.0	1026	57 / 121
drr, 4 thr	0	0	0	4.0	19.0	0	52 / 130
rr, 1 thr	0	18,040	3,449,130	4.0	13.0	1020	250 / 499
drr, 1 thr (suppl.)	0	0	0	4.0	19.0	0	271 / 573



Why long lines get less than their full allowance under rr. A round can only end on a line boundary. With ordinary 60-79 byte lines the leftover is tiny: a medium GET forfeits about 110 bytes in total over its 3 preempted rounds (under 0.5% of Q per round). A 20,000-byte line changes that. About 4.8 KB of ordinary lines separate the long lines, so when the next long line is reached mid-round it cannot fit in what is left, the round ends early and the remainder is forfeited; the round transfers only part of its Q. That happens once per long line: the large file forfeits 20,289 bytes per GET (3,469,419 in a run), which is about 3.4 KB, or 41% of a quantum, at each of its six long lines. The next round then starts with the long line and A14 sends it whole, overrunning Q by about 11.8 KB. rr's accounting is therefore uneven: it under-serves the long-line file before each long line and over-serves it on the A14 round, finishing the file in 13 rounds. **drr recovers exactly what rr forfeits:** the unused part of each allowance becomes deficit, so every round's allowance is Q plus what was left over and the file's cumulative service tracks Q per round. It never forfeits, never fires A14, and needs 19 rounds (= ceil(153,613 / 8192)), of which the deficit-building rounds ahead of each long line transfer nothing. What drr buys is exact, predictable accounting; what it costs is more rounds (and queue trips) for the long-line file.

Effect on response time. The mechanism differences above are large and deterministic; their effect on the large file's response time is small. Its median slowdown is 57 (rr) and 52 (drr) ns/byte at the reference configuration, with overlapping run-to-run ranges (57 [46-65] vs 52 [48-54]), and 250 (rr) vs 271 (drr) with one thread (250 [230-258] vs 271 [260-305]; fig. 3c). With one thread drr's large file is 8% slower in every run: it needs 19 rounds against rr's 13, and each extra round is a trip to the back of a longer queue. The reason the effect is small is load: with 8 closed-loop clients at most 4 requests wait at the reference configuration (7 with one thread), so an extra round, or a zero-progress deficit round, is soon followed by another turn.

5. Trade-offs and the SJF starvation



Who each policy penalises. **fcfs** is fair in waiting (all classes wait alike: 1056 / 1097 / 1111 us at p50) but penalises *small* requests, which queue behind long transfers: their slowdown is the highest in every configuration (16234 ns/byte at one thread). **sjf** reverses this: with one thread it gives the best small and medium slowdown (910 and 182 ns/byte, against 2641 and 390 for rr) and pushes the cost onto the *large* class (294 ns/byte, at or above rr / drr at 250 / 271 and about 2.3x fcfs). At the reference configuration rr / drr already match or beat sjf for small files (826 / 737 vs 979). **rr** and **drr** get a similar small-file gain without ever ranking requests by size: they cut small-file slowdown by about 6x at one thread while charging the large class about 2.0x its fcfs slowdown, because a large request is now interleaved with everyone else instead of running to completion. What separates sjf is waiting time, next.

The SJF starvation (A11). Under sjf the wait depends on size: at the reference configuration the median wait is 402 us for small, 703 for medium and 2592 us for large requests (large waits 6.4x longer than small; under fcfs, rr and drr the three classes wait about the same). The tail shows it: with one thread the large class's p99 waiting time is 81 ms under sjf against 41 ms under fcfs, and sjf has the largest overall p99 waiting time in that configuration (71 ms vs 41 ms). Starvation here is bounded, because each of the 8 clients has at most one request outstanding, so a large request is overtaken only by requests issued while it waits; with an open arrival stream of small requests it would be unbounded, which is why real systems add aging.

6. Aside: --p (A30) and caveats

--p changes system calls, not scheduling. fcfs at the reference configuration with --p 1 vs --p 10 (3 alternating pairs): send() calls fell from about 157,256 to 16,811 per run (9.4x fewer, in every pair). Median waiting was 2341 us with --p 1 and 3207 us with --p 10 (no consistent difference between the pairs; the waiting effect is smaller than the host's noise). Throughput was too noisy to rank (medians 898 vs 830 req/s, single runs 273-1087). The scheduling columns are unaffected (rounds = 1, forfeited_bytes = 0 in both). **Caveats.** One machine, loopback, three workload files and a closed loop of 8 clients: the queue is short, which limits how far the policies can separate. Throughput and tail values move a lot between runs on this host, so the report relies on medians and on comparisons made inside the same block of runs. The A14 reading above is my interpretation of the spec; the alternative (apply it to a long line anywhere in a round) would leave rr with almost no forfeiture on this workload and remove the effect described in section 4.